



Home



Web Development



React Chapter 5

Hooks

Learn React Hooks, especially `useState` and `useEffect`, with simple explanations and examples.

 Did You Know?

Hooks were introduced in React 16.8. They allow functional components to use state and other React features without writing class components.

 What are Hooks?

Hooks are special React functions that let developers use features like State and Lifecycle inside functional components.

 Why do we use Hooks?

- Use State in functional components

- Write cleaner code
- Avoid class components
- Reuse logic easily
- Improve code readability



Common React Hooks

Hook	Purpose
useState()	Stores changing data
useEffect()	Performs side effects
useContext()	Access shared data
useRef()	Access DOM elements



useState Example

```
import { useState } from "react";  
  
function Counter(){  
  
  const [count,setCount] = useState(0);  
  
}
```



useEffect Example

```
import { useEffect } from "react";  
  
useEffect(()=>{  
  
  console.log("Component Loaded");  
  
},[]);
```



Rules of Hooks

- Call Hooks at the top level
- Do not call Hooks inside loops

- Use Hooks only inside React functions
- Always import Hooks before using them

Advantages

- Cleaner code
- Easy state management
- Reusable logic
- Better performance
- Simpler React applications

Common Mistakes

- Calling Hooks inside conditions
- Forgetting to import Hooks
- Using Hooks outside components

- Updating State incorrectly



Interview Questions

Q1: What are Hooks?

Hooks are special React functions used in functional components.

Q2: Which Hook is used for State?

`useState()`

Q3: Which Hook is used for side effects?

`useEffect()`

Q4: When were Hooks introduced?

React 16.8



Quick Revision

Hooks Special React Functions

State Hook `useState()`

Effect Hook `useEffect()`

Released React 16.8



Download React Notes

Download complete React notes PDF and revise concepts anytime.